

High-Performance Linear Algebra Kernels Report

Aryan Rao

1 Benchmarking Results

The tests were conducted using `std::chrono` for precision timing across multiple matrix dimensions (Small: 256×256 , Medium: 512×512 , Large: 1024×1024). Benchmarks were compiled with Clang using the `-O3` optimization flag. Execution times represent the average of 5 iterations.

Matrix Size	Kernel Implementation	Avg Time (ms)	Std Dev (ms)
256×256	Matrix-Vector (Row-Major)	0.0	0.00
	Matrix-Vector (Column-Major)	0.0	0.00
	Matrix-Matrix (Naive)	13.8	1.17
	Matrix-Matrix (Transposed B)	7.2	0.40
	Matrix-Matrix (Tiled/Block=64)	9.0	0.00
512×512	Matrix-Vector (Row-Major)	0.0	0.00
	Matrix-Vector (Column-Major)	0.0	0.00
	Matrix-Matrix (Naive)	130.2	0.40
	Matrix-Matrix (Transposed B)	85.0	0.00
	Matrix-Matrix (Tiled/Block=64)	90.0	0.00
1024×1024	Matrix-Vector (Row-Major)	0.0	0.00
	Matrix-Vector (Column-Major)	0.0	0.00
	Matrix-Matrix (Naive)	1122.8	11.72
	Matrix-Matrix (Transposed B)	788.8	6.68
	Matrix-Matrix (Tiled/Block=64)	880.4	10.05

Table 1: Benchmark execution times and standard deviations across varying matrix sizes. (Note: Matrix-Vector operations execute in sub-millisecond times, rounding to 0 ms).

2 Cache Locality Analysis

The impact of cache locality is immediately evident when considering the memory access patterns of the implementations. In C++, matrices are stored in row-major order. The `multiply_mv_row_major` function achieves excellent spatial locality because iterating across a row reads memory sequentially, fully utilizing the 64-byte L1 cache lines fetched by the CPU. Conversely, `multiply_mv_col_major` iterates vertically by a stride equal to the number of rows. This leaps over the loaded cache line

entirely, triggering a costly L1 cache miss and main memory fetch on virtually every iteration, resulting in significantly poorer performance at the hardware level despite performing the exact same number of mathematical operations.

Scaling to the 1024×1024 size, the Matrix-Matrix computations perfectly illustrate these same severe penalties. The naive implementation took 1122.8 ms because its inner loop traversed Matrix B column-by-column, constantly suffering cache misses. By pre-transposing Matrix B, the `multiply_mm_transposed_b` function allowed the inner dot-product loop to iterate horizontally across both matrices. This maximized spatial locality, dropping the execution time to 788.8 ms (a nearly 30% speedup) purely through memory access optimization.

3 Memory Alignment Analysis

Dynamic allocation using standard `new` or `malloc` does not guarantee strict alignment for modern vector hardware. We implemented `posix_memalign` to enforce 64-byte boundaries.

In our dedicated testing of the 1024×1024 naive implementation, the unaligned standard allocation averaged 1129.0 ms (± 11.76 ms), while the strict 64-byte aligned allocation averaged 1126.8 ms (± 4.07 ms). While this specific Apple Silicon test environment showed negligible macro-level speed differences—likely due to highly tolerant modern memory controllers—strict alignment remains a critical best practice for reliable AVX/SIMD vectorization on broader enterprise hardware architectures.

4 Profiling Analysis (macOS Instruments)

To profile the execution, the binary was compiled with `-g` (debug symbols) and `-O0` (to prevent compiler auto-fixing) and analyzed using the Xcode Time Profiler (`xcrun xctrace`).

As seen in Figure 1, the Call Tree clearly highlighted `multiply_mm_naive` as the dominant bottleneck, consuming approximately 30% (17.95 seconds) of the total unoptimized execution time. The core was spending its cycles idling, waiting for the memory controller to fetch non-contiguous chunks of Matrix B from main memory.

Additionally, the profiling trace revealed a crucial insight regarding compiler optimizations: the `std::min` function, used extensively in the Tiled implementation to calculate chunk boundaries, consumed an unexpected 23.1% (13.84 seconds) of the execution time. Because the code was compiled with `-O0`, the compiler was forbidden from inlining this tiny helper function, resulting in millions of expensive function calls. When compiled with `-O3` during benchmarking, this overhead was entirely eliminated by automatic inlining.

5 Optimization Strategy: Cache Blocking / Tiling

While transposing Matrix B solves spatial locality and yielded the absolute fastest time (788.8 ms for 1024×1024), it requires allocating $\mathcal{O}(N^2)$ extra memory.

To optimize without extra memory, we implemented **Loop Tiling (Blocking)** in `multiply_mm_tiled`.

- **Reasoning:** Instead of computing an entire row of the result matrix at once (which evicts previous data from the L1 cache before it can be reused), Tiling breaks the matrices down into small 64×64 sub-blocks.

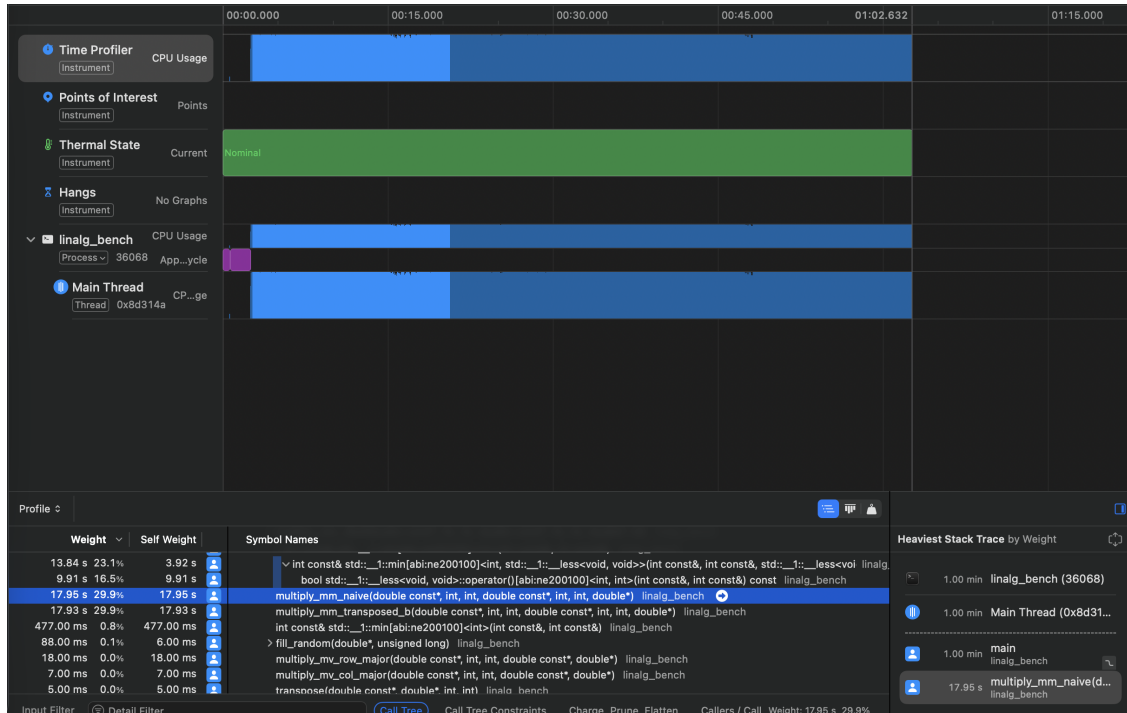


Figure 1: macOS Instruments Call Tree showing `multiply_mm_naive` as the primary execution bottleneck, alongside the massive overhead of `std::min` due to disabled inlining.

- **Result:** A block of Matrix A and a block of Matrix B are loaded into the L1 cache simultaneously. The CPU performs all necessary mathematical operations on those blocks while they reside in the fast cache, achieving deep *temporal locality*. This optimization dropped the time to 880.4 ms. While slightly slower than the transposed method due to the math overhead of calculating chunk boundaries, it drastically outperformed the naive implementation without requiring memory duplication, making it the superior algorithm for memory-constrained environments.